

Push_swap: Diario delle Modifiche

Guida allo studio delle correzioni apportate

1. Risoluzione del Crash (Puntatori Corrotti)

L'errore *"Attempt to extract a component..."* era dovuto a un puntatore `next` non inizializzato che mandava il debugger in tilt dopo il 2° numero.

Correzione in `ft_lstnew (utils.c)`

```
t_node *ft_lstnew(int value) {
    t_node *new = malloc(sizeof(t_node));
    if (!new) return (NULL);
    new->value = value;
    new->index = -1;
    new->next = NULL; // ESSENZIALE: senza questo, ft_lstlast crasha
    return (new);
}
```

2. Ottimizzazione del Parsing (`parse.c`)

Abbiamo riordinato la sequenza per evitare **Memory Leak**. Ora controlliamo i duplicati e i range PRIMA di creare il nodo.

```
// Sequenza sicura:
1. tmp = ft_atoi(argv[i]);
2. if (check_overflow(tmp)) return (error);
3. if (check_duplicate(stack_a, tmp)) return (error);
```

```
4. new_node = ft_lstnew(tmp); // Solo se tutto è OK
5. ft_lstadd_back(&stack_a, new_node);
```

3. Logica dei Chunk (Algoritmo per Grandi Stack)

Il loop infinito era causato da un `while` nel main che non terminava mai e dall'uso di puntatori statici.

Eliminazione dei puntatori "Fantasmi"

Nelle funzioni di movimento, abbiamo smesso di usare variabili `tmp` locali che rimanevano "ferme" mentre lo stack ruotava.

```
// SBAGLIATO:
while (tmp != max) move_rb(stack); // tmp non si aggiorna!

// CORRETTO:
while (aabb->stack_b != max_node) // Controlla sempre la TESTA attuale
    move_rb(aabb);
```

4. Funzioni di Supporto Critiche

get_position

Restituisce la posizione di un nodo. Il `-1` serve come valore di errore se il nodo non viene trovato.

```
int get_position(t_node *stack, t_node *target) {
    int i = 0;
    while (stack) {
        if (stack == target) return (i);
        stack = stack->next;
        i++;
    }
}
```

```
    return (-1);  
}
```

sort_check

Aggiunto il controllo obbligatorio sullo Stack B: il programma non è finito finché B non è vuoto!

 **Pro Tip della 42:** Ricorda sempre di fare `make re` o `make fclean && make` dopo ogni modifica importante. Il Makefile a volte non "sente" i cambiamenti nei file .h o nei piccoli file .c!